Deep Reinforcement Learning for Disturbance-Robust Ball-on-Tray Balancing

ME5418 – Machine Learning in Robotics | Stage 2: Neural Network Code & Report

> This stage delivers every learnable component of the project — the PPO actor and critic (with an optional asymmetric critic), the SAC actor with twin critics, and the losses of both algorithms — written directly against `torch.nn` with no reinforcement-learning framework in between. `demo_network.py` runs a complete forward and backward pass of both algorithms on data from the stage 1 Gym environment, and two supervised experiments show that the networks can learn. The training loops are stage 3.

1. Code Explanation

Stage 1 (environment, PD/LQR baselines, evaluation) is unchanged; stage 2 adds the `networks/` package, a demo and a test file.

File	Content
`networks/actor_critic.py`	PPO: separate actor and critic MLPs, `act` / `evaluate_actions` / `predict`, the asymmetric variant, save and load.
`networks/sac.py`	SAC: tanh-squashed Gaussian actor with state-dependent std, twin Q-networks in one module.
`networks/losses.py`	Clipped PPO loss, SAC critic and actor losses, discounted returns and normalised advantages. Shared by the demo, the tests and stage 3.
`networks/experiments.py`	Behaviour cloning of the LQR controller; comparison of the baseline and the asymmetric critic.
`demo_network.py`	Five printed sections (below); `--bc`, `--critics` and `--realistic` run the experiments.
`tests/test_networks.py`	19 tests: shapes, probability formulas, tanh correction, loss formulas, gradient flow, determinism, save/load, link to the environment.

1.1 Interface and shared backbone

The environment exposes `Box(18,)` observations and `Box(-1, 1, shape=(2,))` actions. The observation is **18-dimensional, not 6**: stage 1 stacks the last three frames of `[x, y, v_x, v_y, pitch, roll]`; a test feeds real `env.reset` / `env.step` observations through the networks and checks every action with `action_space.contains`. Every network is a ReLU MLP with the `(128, 64)` hidden sizes of the proposal. The observation is low-dimensional and already normalised, so there is no convolutional encoder and no input-normalisation layer. Weights are initialised orthogonally with the gains standard for continuous control: `√2` on hidden layers, **`0.01` on the policy outputs** and `1.0` on value and Q outputs. The small policy gain makes the initial action ≈ 0, i.e. "do not change the tilt" — the safe start, where a large random mean would saturate the ±15° tilt limit before learning begins. All constructors take a `seed` that does not disturb torch's global random state.

1.2 PPO actor–critic

- **Separate actor and critic** (10,820 and 10,753 parameters), no shared trunk, so policy and value gradients do not interfere.
- **State-independent `log_std`**, a free parameter (initially −0.5): it decouples exploration from the state and keeps the entropy schedule easy to control.
- **Only the mean is squashed.** `tanh` keeps the mean in `[-1, 1]`; the Gaussian sample is not squashed. Its log-probability is computed for the raw sample, as PPO's importance ratio requires, and only the action sent to `env.step` is clipped (`act` returns both).
- **Asymmetric variant.** `ActorCritic(privileged_dim=3)` also feeds the critic the rolling resistance c_{rr} and the base acceleration from the environment's `info`. The critic is only used in training, so the deployed actor still needs nothing but the observation.

$$L = -E[\, \min(r_t A_t,\ \mathrm{clip}(r_t,\ 1\pm0.2) A_t) \,] + 0.5\, E[(V-R)^2] - 0.01\, E[H(\pi)]$$

1.3 SAC actor and twin critics

SAC needs the density of the action it applies, so the Gaussian sample `u` is squashed through `tanh` into `(-1, 1)` and the log-probability is corrected by the log-determinant of that change of variables, $\log \pi(a|s) = \log N(u) - \sum_i \log(1 - \tanh^2 u_i)$, evaluated in the stable form $2(\log 2 - u - \mathrm{softplus}(-2u))$; the direct form loses all precision once $|u|$ exceeds about 8. Here the standard deviation *is* a network output, clamped to `[-20, 2]` in log-space (10,948 parameters). `SACCritic` holds two independently initialised Q-networks of input width `18 + 2 = 20` (22,018 parameters); the critic loss regresses both to $y = r + \gamma(1-d)[\min(Q_1', Q_2') - \alpha \log \pi(a'|s')]$, the actor loss is $E[\alpha \log \pi(\tilde{a}|s) - Q_1(s, \tilde{a})]$ with a reparameterised sample, $\alpha = 0.2$.

1.4 What the demo and the experiments show

`demo_network.py` (1) builds all networks, (2) runs PPO forward on a real `env.reset()` observation, (3) collects a 64-step rollout *in the environment*, forms the clipped PPO loss, calls `.backward()`, prints the **gradient norm of every parameter tensor** and takes one Adam step (total loss 19.235 → 19.129), (4) does the same for the SAC critic and actor losses on these transitions and checks the tanh correction against the direct formula (difference 4×10^{-7}), and (5) lets the untrained policy drive one episode. Two supervised experiments show the networks can *learn*, using noisy-LQR rollouts (training seeds 1000–1199, validation 2000–2049):

Experiment	Result (100 evaluation seeds / 3 network seeds)
Behaviour cloning of the LQR	The PPO actor, fitted to LQR actions, drives the tray as well as the LQR itself: return `724.3 ± 8.5` vs `724.4 ± 8.3`, 100 % survival for both. With noise, lag and delay: 616.0 (94 %) vs 627.4 (97 %).
Asymmetric vs baseline critic	On the same discounted returns, the critic with c_{rr} and a_{base} has a 24 % lower validation error (20 % in the realistic setting, 38 % during disturbance pulses): the hidden quantities carry information about the return.

2. Existing Codes and Libraries

PyTorch 1.13.1 is the only external machine-learning dependency. We use `nn.Linear`, `nn.ReLU`, `nn.init.orthogonal_`, `torch.distributions.Normal`, `torch.optim.Adam` and autograd. Everything above that level — the architectures, the initialisation scheme, the Gaussian and tanh-squashed policy heads, the log-probability corrections, the twin-critic module and all losses — is written in this project. **We deliberately did not use an RL framework** (stable-baselines3, RLlib, Tianshou): it would hide exactly the parts this stage is about, and current stable-baselines3 requires `gymnasium`, while our environment targets `gym==0.26.2`. The environment itself is ours and pure numpy: **NO EXTERNAL ROBOTIC SIMULATOR** is needed; `matplotlib` only draws.

What we had to do to make it work. Stage 1 is pinned to Python 3.7, and PyTorch 1.13.1 is the last release with Python 3.7 wheels, so it keeps the whole project on one interpreter. `environment.yml` takes torch from PyTorch's CPU wheel index, which avoids the much larger CUDA build on Linux; the unused `moviepy` dependency was dropped (GIFs are written with Pillow). **References:** the clipped surrogate follows Schulman et al. (2017), the tanh correction and twin-critic target Haarnoja et al. (2018); we re-derived and unit-tested these instead of copying an implementation (e.g. `test_sac_tanh_correction_is_correct`, `test_ppo_loss_matches_formula`).

3. Reflections and Lessons Learned

3.1 Changes from the proposal

- **The observation is 18-D, not 6-D.** Neither c_{rr} nor the base acceleration is observable, so the task is a POMDP; three stacked frames let a feedforward policy infer them. Only the input layer widened, so the proposal's "no recurrent network needed" argument still holds, now for a concrete reason.
- **Action bounding differs per algorithm**, which the proposal did not specify: PPO bounds only the mean and lets the environment clip the action; SAC squashes the sample and pays the Jacobian correction. They are not interchangeable — this only became clear while writing the code.
- **PPO's standard deviation is a free parameter**, not an output as the proposal described (stability, §1.2); only SAC uses a state-dependent std.
- **Added: the asymmetric critic.** The simulator knows the quantities the robot cannot sense; giving them to the critic only is a cheap way to use them, and the experiment shows they help.

3.2 Assumptions and simplifications at this stage

The demo uses plain discounted returns, not GAE(λ); the SAC demo uses the critic itself as its target (training will use a slowly updated copy); the SAC temperature is fixed at $\alpha = 0.2$.

3.3 Lessons

- **The tanh correction raises the log-probability.** Our first test asserted the opposite. `tanh` compresses $\mathbb{R}$ onto `(-1,1)`, so the squashed density is higher and $\log(1-\tanh^2 u) \le 0$ is *subtracted*. Asserting that the stable and the direct form agree exposed it.
- **A zero PPO policy loss on the first epoch is correct.** The rollout comes from the current weights, so the ratio is 1 and the normalised advantages average to zero, while the *gradient* is non-zero. The demo says so, because it reads like a bug.
- **A reparameterised loss can only be compared before and after an update with the same noise.** Our first SAC readout showed the actor loss *rising* after a descent step; re-seeding before each evaluation showed the real effect (−0.294976 → −0.295296).
- **A backward pass is not evidence of learning.** One optimiser step shows the graph is wired; behaviour cloning to LQR-level closed-loop performance shows the actor can represent a good policy.
- **Keep one code base.** Two network versions were first written in parallel on different stage 1 snapshots; merging them onto the current one and re-running all 48 tests in a fresh environment was easy only because the environment interface had not changed.

3.4 Remaining changes towards the final version

Stage 3 adds the training loops: GAE(λ), minibatch epochs and learning-rate annealing for PPO; a replay buffer, target critics and automatic entropy tuning for SAC; and the staged domain randomisation of the proposal, ending with the realism parameters (noise, lag, delay). Whether the asymmetric critic speeds up PPO is an open question for that stage.

4. How to Run

```
conda env create -f environment.yml && conda activate me5418-ballontray
python demo_network.py [--variant asymmetric] [--seed N]   # five sections, PPO and SAC
python demo_network.py --bc | --critics [--realistic]       # the two experiments, about 1 min each
python tests/test_networks.py                               # 19 network tests
python demo.py                                              # stage 1: 3D demo, random then LQR
```

Verification: in a fresh environment created from `environment.yml` (Windows 11, Python 3.7.16, torch 1.13.1+cpu, numpy 1.21.6, gym 0.26.2, matplotlib 3.5.3), all 48 tests pass (14 environment, 7 renderer, 8 realism, 19 network) and every demo option runs; two runs print identical numbers. CPU suffices: the largest network has 22k parameters. `README.md` documents every file and argument.